Functional Programming in Python

Writing Clearer, Safer Code with Pure Functions and Higher-Order Tools

FUNCTIONAL MINDSET

The Functional Mindset

- Functional programming treats computation as evaluating functions, avoiding shared, mutable state.
- Python is not purely functional, but supports functional techniques alongside OOP and procedural styles.
- Favour functions that return new values over functions that mutate their inputs.
- This style makes code easier to test, reason about, and eventually run in parallel.

01_functional_mindset.py

```python
def add_tax(price, rate):
    return price * (1 + rate)

# Same inputs always produce the same output
print(round(add_tax(100, 0.16), 2))   # 116.0
print(round(add_tax(100, 0.16), 2))   # 116.0 again, guaranteed
```

CORE CONCEPT

Pure Functions & Side Effects

- A pure function's output depends only on its inputs, and it changes nothing outside itself.
- A side effect is any change visible outside the function: a global, a file, a print, a mutated argument.
- Pure functions are easy to test in isolation, and safe to reorder, cache, or run concurrently.
- Impure code still has its place (I/O, logging) but is best kept separate from core logic.

02_pure_functions.py

```python
total_calls = 0

def impure_add(a, b):
    global total_calls
    total_calls += 1        # side effect: mutates external state
    return a + b

def pure_add(a, b):
    return a + b            # no side effect, same result always

print(pure_add(2, 3))       # 5
print(pure_add(2, 3))       # 5, no hidden state involved
```

ANONYMOUS FUNCTIONS

Lambda Functions

- A lambda is a small, unnamed function: lambda arguments: expression.
- Limited to a single expression; no statements, no assignments, no multiple lines.
- Commonly used where a short function is needed only once, e.g. as an argument.
- For anything longer, or reused elsewhere, a normal def function is clearer.

03_lambda_functions.py

```python
square = lambda x: x ** 2
print(square(5))            # 25

# Common use: as a sort key
people = [("Amy", 34), ("Bo", 22), ("Cy", 29)]
people_by_age = sorted(people, key=lambda person: person[1])
print(people_by_age)
# [('Bo', 22), ('Cy', 29), ('Amy', 34)]
```

HIGHER-ORDER FUNCTIONS

map(): Transforming a Sequence

- map(function, iterable) applies function to every item, returning a lazy iterator.
- Replaces a manual for-loop that only builds a new list from an old one.
- Wrap in list(…) to see or store the results eagerly.
- Works with several input iterables at once, of matching length.

04_map_transform.py

```
prices = [10, 20, 30]

with_tax = map(lambda p: p * 1.16, prices)
print(list(with_tax))       # [11.6, 23.2, 34.8]

# map with two iterables
quantities = [2, 1, 5]
totals = list(map(lambda p, q: p * q, prices, quantities))
print(totals)               # [20, 20, 150]
```

HIGHER-ORDER FUNCTIONS

filter(): Selecting Elements

- filter(function, iterable) keeps only the items where function returns True.
- The function should be a predicate: it inspects one item and returns a bool.
- Passing None as the function filters out falsy values (0, '', None, False).
- Like map, filter returns a lazy iterator, not a list.

05_filter_select.py

```
numbers = [1, -4, 7, 0, -2, 9]

positives = filter(lambda n: n > 0, numbers)
print(list(positives))      # [1, 7, 9]

# filter(None, ...) drops falsy values
mixed = [0, 1, "", "hi", None, 5]
print(list(filter(None, mixed)))
# [1, 'hi', 5]
```

HIGHER-ORDER FUNCTIONS

reduce(): Aggregating a Sequence

- functools.reduce(function, iterable, initial) folds a sequence down to a single value.
- function takes an accumulator and the next item, returning the new accumulator.
- Provide an initial value to define behaviour on an empty sequence.
- Useful for totals, products, or building up one combined result.

06_reduce_aggregate.py

```python
from functools import reduce

prices = [10, 20, 30]
total = reduce(lambda acc, p: acc + p, prices, 0)
print(total)                # 60

# Same idea, finding the maximum
values = [4, 9, 2, 7]
largest = reduce(lambda acc, v: v if v > acc else acc, values)
print(largest)              # 9
```

HIGHER-ORDER FUNCTIONS

Higher-Order Functions & partial

- A higher-order function takes another function as an argument, or returns one.
- map, filter, reduce, and sorted's key argument are all higher-order functions.
- functools.partial fixes some arguments of a function, producing a new, simpler function.
- Partial application avoids writing a wrapper function just to preset one argument.

07_partial_higher_order.py

```python
from functools import partial

def power(base, exponent):
    return base ** exponent

square = partial(power, exponent=2)
cube = partial(power, exponent=3)

print(square(5))            # 25
print(cube(2))               # 8
```

COMPOSITION

Composing map, filter, reduce

- Functional tools chain naturally: filter first, then map, then reduce.
- Each stage stays small and single-purpose, which keeps pipelines readable.
- Lazy iterators mean nothing is computed until the final reduce forces evaluation.
- For everyday code, a list comprehension is often more readable than chaining map/filter.

08_composing_pipelines.py

```python
from functools import reduce

orders = [15, -3, 42, 8, -1, 23]

pipeline = reduce(
    lambda acc, n: acc + n,
    map(lambda n: n * 1.16,
        filter(lambda n: n > 0, orders)),
    0
)
print(round(pipeline, 2))    # 102.08

# The same result, written as a comprehension
total = sum(n * 1.16 for n in orders if n > 0)
print(round(total, 2))       # 102.08
```

PANDAS INTEGRATION

Lambda Functions in pandas

- apply() runs a lambda across a Series or DataFrame, row by row or element by element.
- Lambdas keep a one-off transformation inline, without defining a named function.
- assign() combines naturally with lambda to add computed columns in a chain.
- For simple element-wise math, a vectorised expression usually beats apply with a lambda.

09_lambda_in_pandas.py

```python
import pandas as pd

df = pd.DataFrame({
    "name": ["Amy", "Bo", "Cy"],
    "price": [10, 20, 30],
})

df["price_with_tax"] = df["price"].apply(lambda p: p * 1.16)

df = df.assign(
    is_expensive=lambda d: d["price"] > 15
)
print(df)
#   name  price  price_with_tax  is_expensive
# 0  Amy     10            11.6         False
# 1   Bo     20            23.2          True
# 2   Cy     30            34.8          True
```

WHY IT MATTERS

Benefits of Functional Programming

1

Predictability

Pure functions always behave the same way, making bugs easier to trace.

2

Testability

No hidden state means a function can be tested with plain input/output checks.

3

Composability

Small functions combine into pipelines without tangled dependencies.

4

Safer Concurrency

Functions without shared mutable state avoid a whole class of race conditions.

5

Readability

map/filter/reduce describe what happens, not the mechanics of how to loop.

SUMMARY

Key Takeaways

1

Prefer Pure Functions

Same inputs, same outputs, no hidden side effects.

2

Use lambda Sparingly

Reach for lambda for short, throwaway functions; use def for anything named or reused.

3

map / filter / reduce

Transform, select and aggregate; comprehensions often read more clearly.

4

functools.partial

Pre-fills arguments, turning a general function into a specialised one.

5

Lambdas in pandas

Convenient inside apply/assign, but vectorised operations scale better.